

Multiple linear regression

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [3]: df_index=pd.read_csv('/Users/mujeebmohammed/Downloads/economic_index.csv')
df_index.head()
```

```
Out[3]:
```

	Unnamed: 0	year	month	interest_rate	unemployment_rate	index_price
0	0	2017	12	2.75	5.3	1464
1	1	2017	11	2.50	5.3	1394
2	2	2017	10	2.50	5.3	1357
3	3	2017	9	2.50	5.3	1293
4	4	2017	8	2.50	5.4	1256

```
In [4]: #drop unnecessary columns
df_index.drop(columns=['Unnamed: 0','year','month'],inplace=True)
df_index.head()
```

```
Out[4]:
```

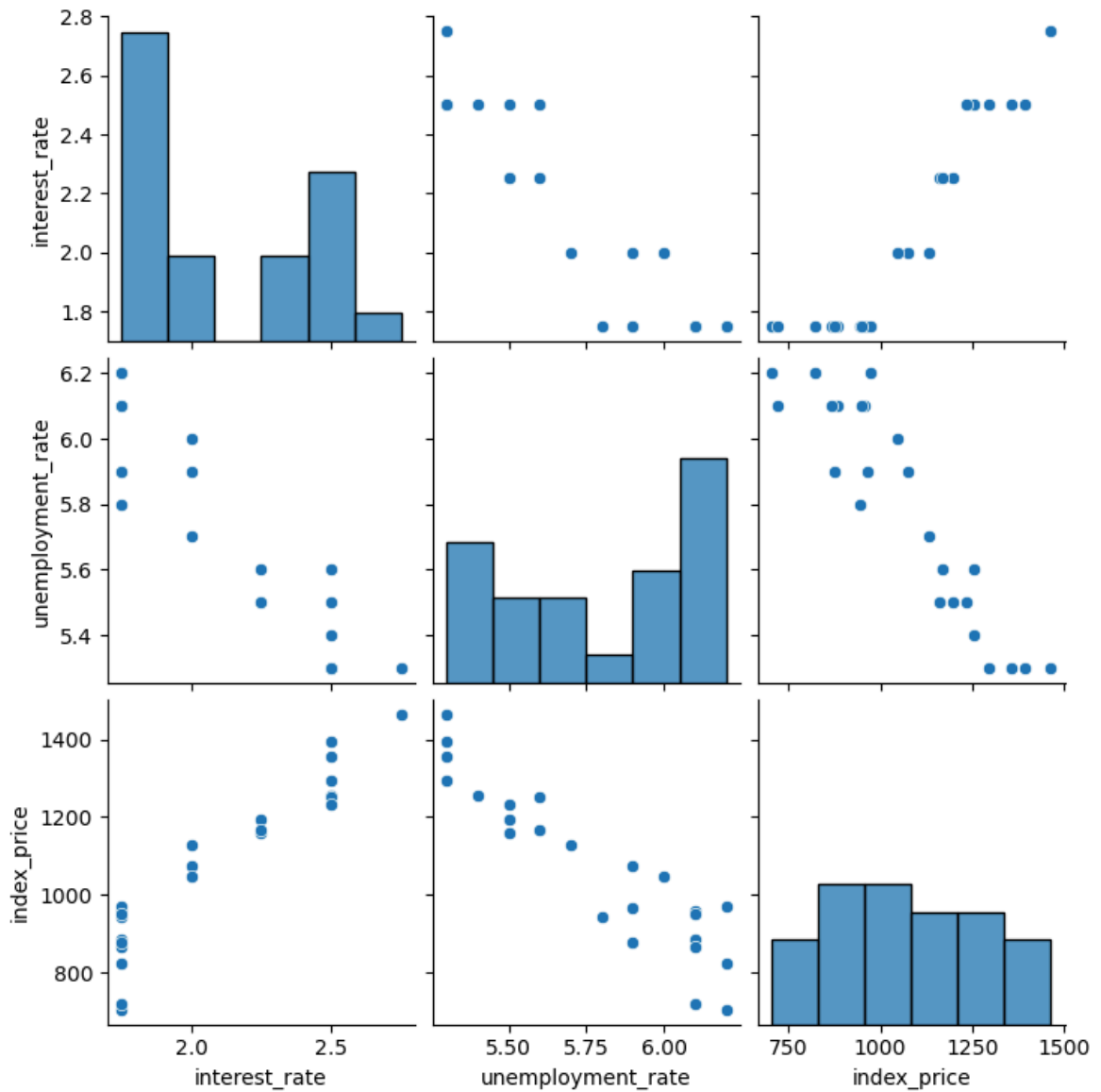
	interest_rate	unemployment_rate	index_price
0	2.75	5.3	1464
1	2.50	5.3	1394
2	2.50	5.3	1357
3	2.50	5.3	1293
4	2.50	5.4	1256

```
In [5]: #check null values
df_index.isnull().sum()
```

```
Out[5]: interest_rate      0
unemployment_rate      0
index_price            0
dtype: int64
```

```
In [6]: #visualization
sns.pairplot(df_index)
```

```
Out[6]: <seaborn.axisgrid.PairGrid at 0x1317036b0>
```



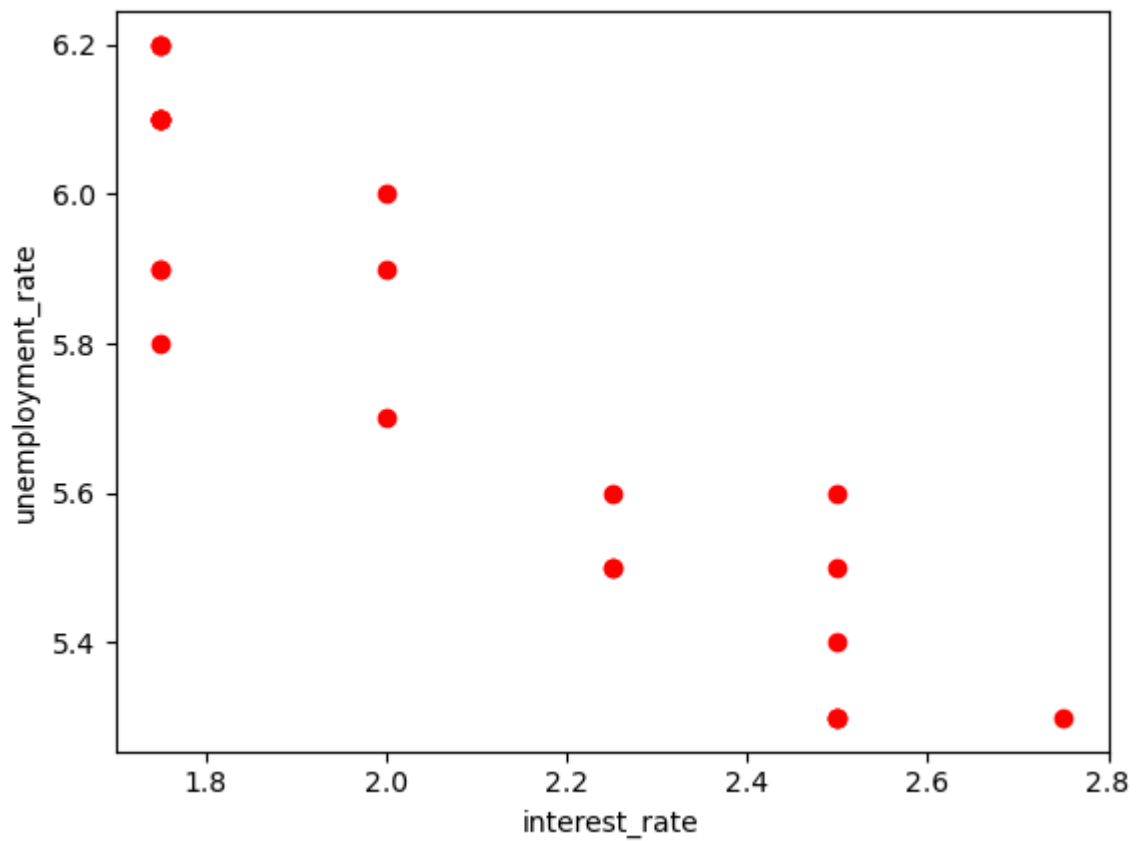
```
In [7]: df_index.corr()
```

```
Out[7]:
```

	interest_rate	unemployment_rate	index_price
interest_rate	1.000000	-0.925814	0.935793
unemployment_rate	-0.925814	1.000000	-0.922338
index_price	0.935793	-0.922338	1.000000

```
In [8]: #visualize more closely
plt.scatter(df_index['interest_rate'],df_index['unemployment_rate'],color
plt.xlabel('interest_rate')
plt.ylabel('unemployment_rate')
```

```
Out[8]: Text(0, 0.5, 'unemployment_rate')
```



```
In [9]: #independent and dependent features
```

```
X=df_index.iloc[:, :-1]  
y=df_index.iloc[:, -1]
```

```
In [10]: X.head()
```

```
Out[10]:
```

	interest_rate	unemployment_rate
0	2.75	5.3
1	2.50	5.3
2	2.50	5.3
3	2.50	5.3
4	2.50	5.4

```
In [11]: y
```

```
Out[11]: 0      1464
         1      1394
         2      1357
         3      1293
         4      1256
         5      1254
         6      1234
         7      1195
         8      1159
         9      1167
        10      1130
        11      1075
        12      1047
        13       965
        14       943
        15       958
        16       971
        17       949
        18       884
        19       866
        20       876
        21       822
        22       704
        23       719
        Name: index_price, dtype: int64
```

```
In [12]: #train test split

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.25,random_
```

```
In [13]: from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.fit_transform(X_test)
```

```
In [14]: X_train
```

```
Out[14]: array([[ -0.90115511,  0.37908503],
                 [ 1.31077107, -1.48187786],
                 [-0.90115511,  1.30956648],
                 [ 1.31077107, -0.55139641],
                 [ 1.31077107, -1.48187786],
                 [-0.16384638,  0.68924552],
                 [-0.90115511,  0.999406  ],
                 [ 1.31077107, -1.48187786],
                 [ 1.31077107, -1.17171738],
                 [-0.90115511,  1.30956648],
                 [-0.90115511,  0.999406  ],
                 [-0.90115511,  0.37908503],
                 [-0.90115511,  0.999406  ],
                 [ 0.57346234, -0.8615569 ],
                 [-0.16384638, -0.24123593],
                 [-0.90115511,  0.06892455],
                 [-0.90115511,  0.999406  ],
                 [ 1.31077107, -0.8615569 ]])
```

```
In [15]: from sklearn.linear_model import LinearRegression
regression=LinearRegression()
```

```
regression.fit(X_train,y_train)
```

Out[15]:

▼ LinearRegression ⓘ ?

LinearRegression()

In [16]:

```
#cross validation
from sklearn.model_selection import cross_val_score
validation_score=cross_val_score(regression,X_train,y_train,scoring='neg_
np.mean(validation_score)
```

Out[16]:

-5649.115244350758

In [17]:

```
#prediction
y_pred=regression.predict(X_test)
y_pred
```

Out[17]:

array([1180.7466813 , 802.74279699, 1379.83457045, 838.52599602,
773.85313963, 1144.96348227])

In [18]:

```
#performance metrics
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score
mse=mean_squared_error(y_test,y_pred)
mae=mean_absolute_error(y_test,y_pred)
rmse=np.sqrt(mse)
score=r2_score(y_test,y_pred)
print(mse,mae,rmse,score)
```

8108.567426306604 73.80444932337097 90.04758423359621 0.7591371539010257

In [19]:

```
#display adjusted R-square
1-(1-score)*len(y_test)/(len(y_test)-X_test.shape[1]-1)
```

Out[19]:

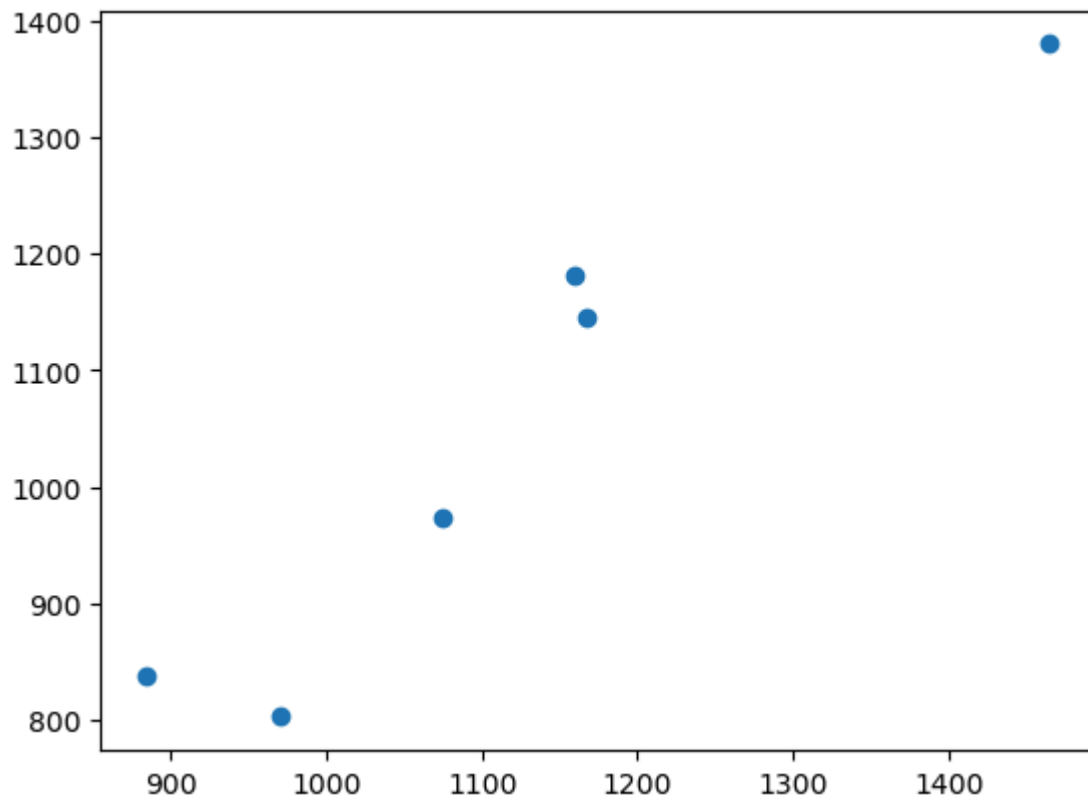
0.5182743078020513

In [20]:

```
#assumptions
plt.scatter(y_test,y_pred)
```

Out[20]:

<matplotlib.collections.PathCollection at 0x1327473e0>

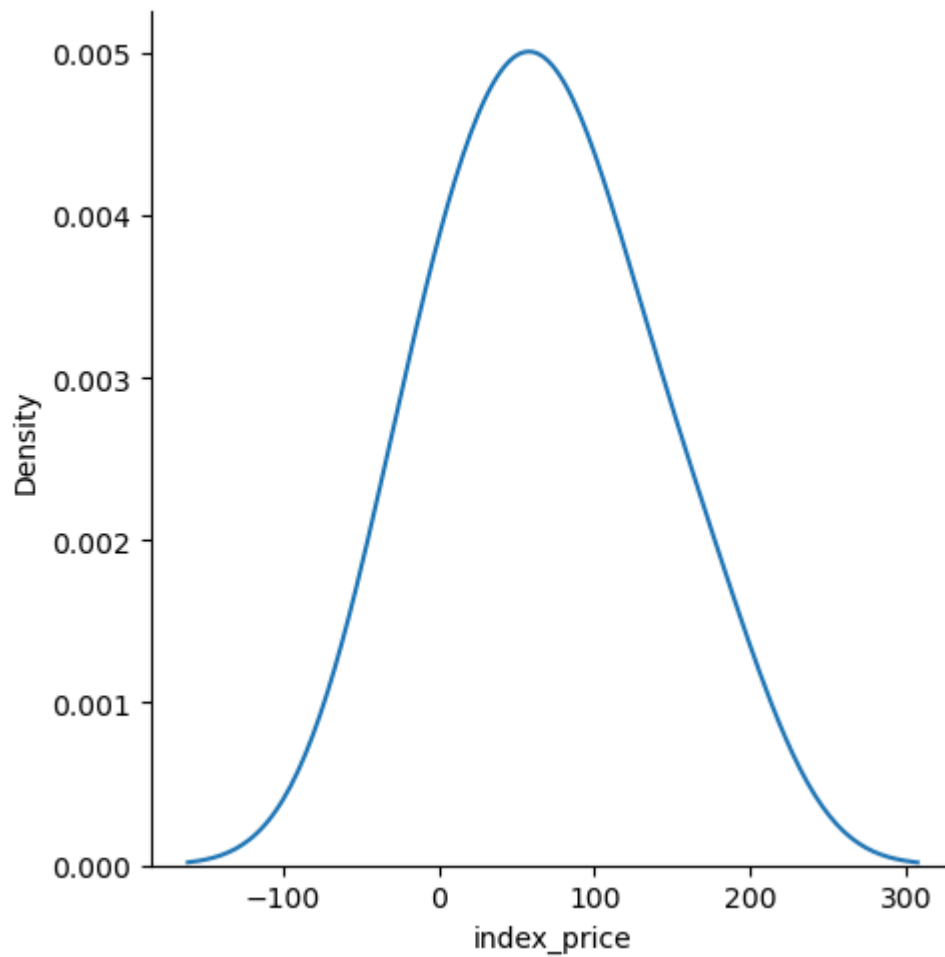


```
In [21]: residuals=y_test-y_pred  
print(residuals)
```

```
8      -21.746681  
16     168.257203  
0       84.165430  
18      45.474004  
11     101.146860  
9       22.036518  
Name: index_price, dtype: float64
```

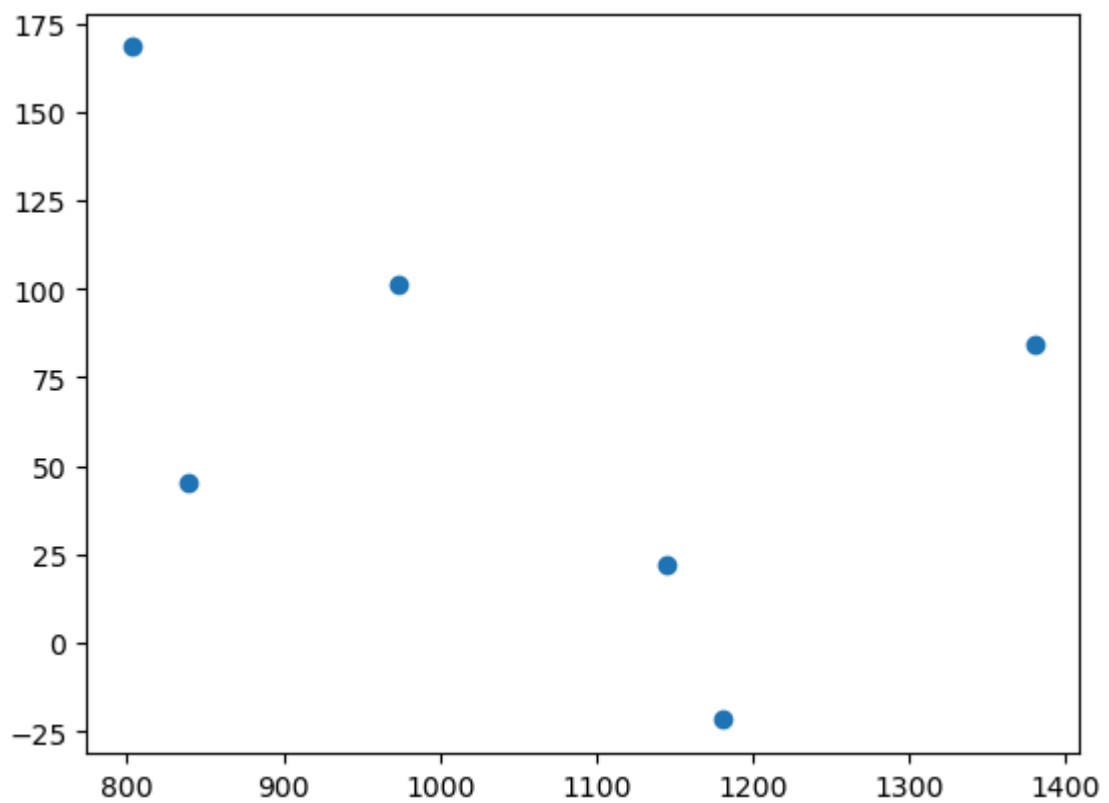
```
In [22]: #plot residuals  
sns.displot(residuals,kind='kde')
```

```
Out[22]: <seaborn.axisgrid.FacetGrid at 0x131b9a120>
```



```
In [23]: #scatter plot with respect to predictions and residuals  
plt.scatter(y_pred,residuals)
```

```
Out[23]: <matplotlib.collections.PathCollection at 0x132f31520>
```



```
In [59]: #OLS linear regression
import statsmodels.api as sm
X_train_sm = sm.add_constant(X_train)
model = sm.OLS(y_train, X_train_sm).fit()
```

```
In [61]: model.summary()
```

```
/Users/anaconda3/lib/python3.12/site-packages/scipy/stats/_axis_nan_policy.py:531: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=18
res = hypotest_fun_out(*samples, **kwds)
```

Out [61]:

OLS Regression Results

Dep. Variable:	index_price	R-squared:	0.903			
Model:	OLS	Adj. R-squared:	0.890			
Method:	Least Squares	F-statistic:	69.88			
Date:	Fri, 27 Mar 2026	Prob (F-statistic):	2.50e-08			
Time:	18:38:20	Log-Likelihood:	-100.85			
No. Observations:	18	AIC:	207.7			
Df Residuals:	15	BIC:	210.4			
Df Model:	2					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	1053.4444	16.941	62.183	0.000	1017.336	1089.553
x1	88.2728	42.263	2.089	0.054	-1.810	178.355
x2	-116.2572	42.263	-2.751	0.015	-206.340	-26.175
Omnibus:	0.598	Durbin-Watson:	1.804			
Prob(Omnibus):	0.741	Jarque-Bera (JB):	0.567			
Skew:	-0.361	Prob(JB):	0.753			
Kurtosis:	2.517	Cond. No.	4.78			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In []: